Integración numérica explicada con un columpio

Milestones 1–3, archivo por archivo, siempre aplicados al oscilador armónico

AM1 – Grupo 1 (MUSE, UPM)

23 de octubre de 2026

Resumen

Este documento recorre todos los archivos de `Milestone_1`, `Milestone_2` y `Milestone_3`. En cada uno se explica primero **la idea**, con su significado físico y una analogía, luego **las matemáticas justas**, y por último **qué hace el código Python**. Todo se aplica siempre al mismo problema, un muelle o columpio sin rozamiento, porque conocemos su solución exacta y podemos ver con nuestros propios ojos qué hace bien y qué hace mal cada método. Las figuras y los números salen de ejecutar el código del repositorio.

Índice

1. El protagonista: el oscilador armónico **2**
- 1.1. Qué es, físicamente . . . 2
- 1.2. La foto del estado: posición y velocidad . . . 2
- 1.3. El plano de fases: un mapa donde la energía es la distancia al centro . . . 2
- 1.4. La vara de medir: ¿cuánto se aleja y cuánto atrasa? . . . 3

2. Milestone 1: tres métodos escritos a mano **5**
- 2.1. `Problema_1.py`: Euler explícito . . . 5
- 2.2. `Problema_2.py`: Runge–Kutta de orden 4 (RK4) . . . 6
- 2.3. `Problema_2_2.py`: el método del trapecio (Crank–Nicolson) resuelto "a tanteo" . . . 9

3. Milestone 2: ordenar el código en piezas intercambiables **11**
- 3.1. `Esquema_fisico.py`: la física . . . 11
- 3.2. `Esquemas_temporales.py`: los cuatro métodos como funciones . . . 12
- 3.3. `Esquemas_temporales_2.py`: el método de Newton . . . 14
- 3.4. `Integradores.py`: la consola, o el bucle de tiempo . . . 16
- 3.5. `Ej_uso_Esquemas.py`: la carrera de los cuatro métodos . . . 17

4. Milestone 3: ¿cuánto me estoy equivocando? **18**
- 4.1. `Esquema_fisico.py`: la física, ya corregida . . . 18
- 4.2. `Non_Linear_Solvers.py`: Newton en su propio archivo . . . 19
- 4.3. `Linear_Solvers.py`: el ajuste a ciegas, ya corregido . . . 19
- 4.4. `Esquemas_temporales.py`: los cuatro métodos, ahora bien . . . 20
- 4.5. `Integradores.py`: la consola, con carta de restaurante . . . 20
- 4.6. `Richardson.py`: estimar el error sin conocer la respuesta . . . 21
- 4.7. `Cauchy_Problem.py`: medir el orden en la práctica . . . 22
- 4.8. `Graphic.py`: los dibujos . . . 23
- 4.9. `Ej_uso_Esquemas.py`: el experimento final . . . 24

5. Resumen: la personalidad de cada método **25**

1. El protagonista: el oscilador armónico

1.1. Qué es, físicamente

Imagina una masa unida a un muelle, o un columpio con oscilaciones pequeñas. Si la separas de su posición de equilibrio, el muelle tira de ella hacia el centro con una fuerza proporcional a lo lejos que está. Escogiendo unidades cómodas:

$$\ddot{x} = -x \qquad \text{(aceleración} = -\text{posición)}.$$

Si está a la derecha ($x > 0$) la empuja a la izquierda, y si está a la izquierda la empuja a la derecha. El resultado es un vaivén eterno, porque no hay rozamiento.

1.2. La foto del estado: posición y velocidad

Para predecir el futuro del columpio no basta con saber *dónde* está. También hay que saber *a qué velocidad* va: en el mismo punto puede estar subiendo o bajando. Así que el estado del sistema es una "foto" con dos datos:

$$\mathbf{U} = \begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} \text{posición} \\ \text{velocidad} \end{pmatrix}.$$

La ley física se convierte en una **regla de cambio F**, que dice: "si ahora estás en este estado, así es como estás cambiando":

$$\frac{d\mathbf{U}}{dt} = \mathbf{F}(\mathbf{U}, t), \qquad \mathbf{F}(\mathbf{U}, t) = \begin{pmatrix} v \\ -x \end{pmatrix} \quad \Longleftrightarrow \quad \begin{cases} \dot{x} = v & \text{(la posición cambia según la velocidad)} \\ \dot{v} = -x & \text{(el muelle cambia la velocidad)} \end{cases}$$

Empezamos con el columpio apartado y quieto: $\mathbf{U}_0 = (x_0, v_0) = (1, 0)$. Saber dónde empiezas y conocer la regla de cambio es lo que se llama un **problema de Cauchy** (o de valor inicial).

Analogía: el navegante sin GPS

Un marinero antiguo, sin GPS, hace *navegación a estima*. Sabe dónde está, conoce su rumbo y su velocidad, y cada cierto tiempo Δt apunta en la carta "si he ido a esta velocidad durante este rato, ahora estaré aquí". Todos los métodos de este curso son variantes de este navegante:

- $\mathbf{U}^n$ es la posición apuntada en la carta en el instante t_n.
- $\mathbf{F}$ es la brújula y el velocímetro: te dicen hacia dónde y cuánto te mueves *ahora*.
- Δt es cada cuánto tiempo actualizas la carta.
- El *cuaderno de bitácora* es la matriz `U` del código, con una fila por instante.

El problema es que el rumbo cambia mientras navegas. Cada método se distingue por *cómo* adivina el rumbo durante el tramo.

1.3. El plano de fases: un mapa donde la energía es la distancia al centro

Si dibujamos la posición en el eje horizontal y la velocidad en el vertical, cada instante es un punto. La solución exacta es

$$x(t) = \cos t, \qquad v(t) = -\sin t,$$

que es una **circunferencia de radio 1** recorrida en sentido horario (Fig. 1). ¿Por qué un círculo? Por la energía:

$$E = \underbrace{\tfrac{1}{2}x^2}_{\text{energía del muelle}} + \underbrace{\tfrac{1}{2}v^2}_{\text{energía de movimiento}} \qquad \Longrightarrow \qquad x^2 + v^2 = 2E = \text{constante}.$$

Idea clave

En este mapa, **la distancia al centro mide la energía** y **el ángulo indica en qué momento del vaivén estamos**. Cuando el columpio está en un extremo, toda la energía está en el muelle ($v = 0$, puntos $(\pm 1, 0)$). Cuando pasa por el centro, toda está en movimiento ($x = 0$). La suma no cambia nunca, así que el punto no se acerca ni se aleja del centro: gira.

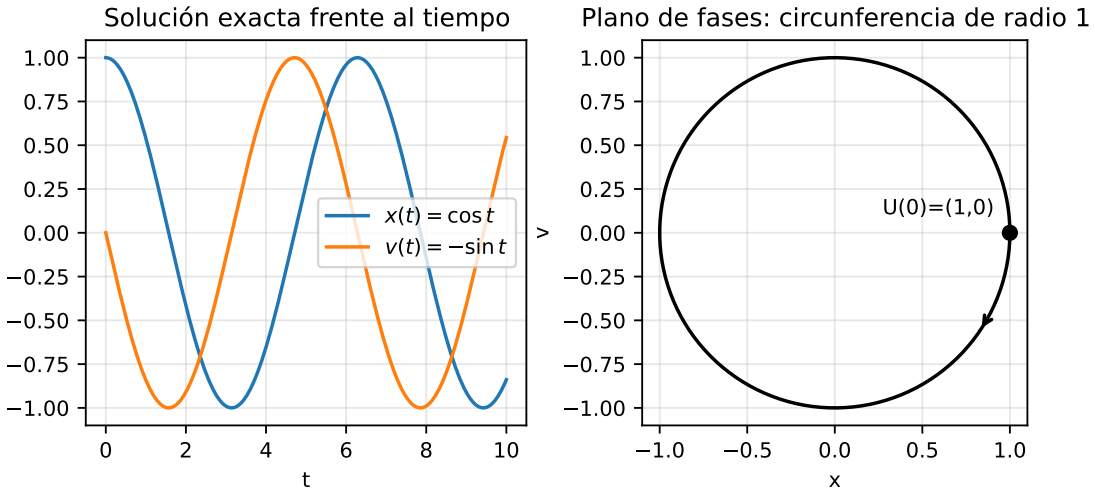


Figura 1: Solución exacta. Izquierda: posición y velocidad oscilan desfasadas. Derecha: en el plano de fases es un círculo, que es lo que debe reproducir un buen método.

1.4. La vara de medir: ¿cuánto se aleja y cuánto atrasa?

Avanzar exactamente un tiempo Δt equivale a **girar el punto un ángulo Δt sin cambiar su distancia al centro**. Aplicado al oscilador, cualquier método de este curso hace en cada paso lo siguiente:

$$\text{multiplicar el radio por } \rho \quad \text{y} \quad \text{girar un ángulo } \theta.$$

Con dos números describimos la "personalidad" de cada método:

Resultado	En el mapa	Significado físico
$\rho > 1$	espiral hacia fuera	aparece energía de la nada: un "empujón fantasma"
$\rho < 1$	espiral hacia dentro	desaparece energía: un "rozamiento fantasma"
$\rho = 1$	se queda en el círculo	la energía se conserva, como en la realidad
$\theta < \Delta t$	va con retraso	un reloj de péndulo que *atrasa*

Tras n pasos, el radio es ρ^n. Si ρ es solo un poco mayor que 1, por ejemplo 1.005, en 100 pasos ya es $1.005^{100} \approx 1.65$. Es el efecto **bola de nieve**: un error pequeño repetido muchas veces se hace grande.

Estabilidad, en una frase. Un método es *estable* para un problema si los errores no hacen bola de nieve, es decir, si $\rho \leq 1$. La Fig. 2 muestra la zona estable de cada método. El oscilador "vive" sobre el eje vertical (imaginario) de esas figuras. Para entenderlas no hace falta saber números complejos: basta con fijarse en si el eje vertical cae dentro (estable) o fuera (inestable) de la zona sombreada.

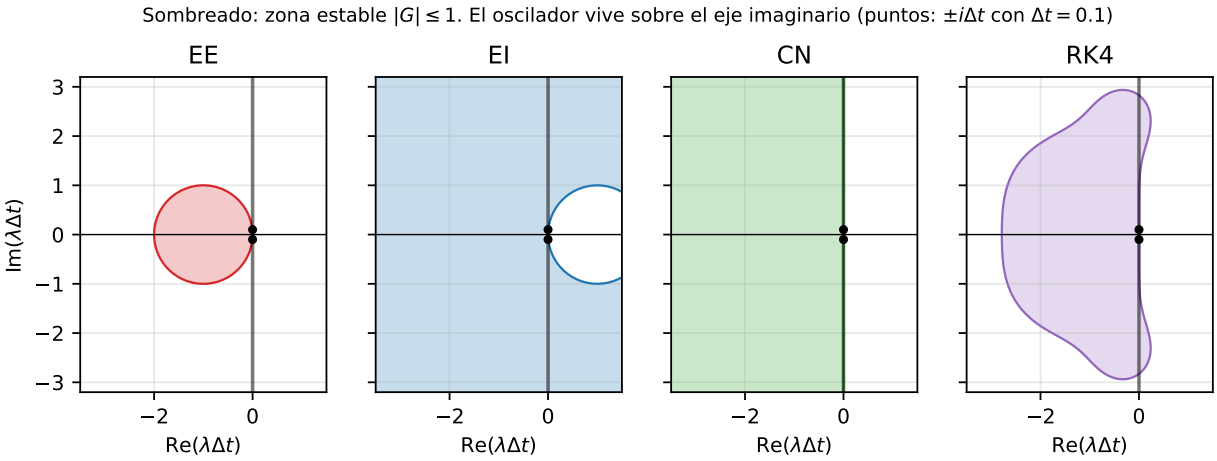


Figura 2: Zonas estables de los cuatro métodos. EE: el eje vertical queda fuera, así que la espiral va siempre hacia fuera. EI: queda dentro con margen, así que siempre pierde energía. CN: el eje es justo la frontera, así que conserva la energía exactamente. RK4: el eje queda dentro pero rozando el borde, así que pierde energía de forma casi imperceptible.

Orden de un método: ¿cuánto mejora si doy pasos más cortos? Si divides el paso Δt entre 2:

- un método de **orden 1** divide el error entre 2;
- uno de **orden 2** lo divide entre $2^2 = 4$;
- uno de **orden 4** lo divide entre $2^4 = 16$.

Analogía

Dibujar un círculo con segmentos rectos: con más segmentos, más cortos, el polígono se parece más al círculo. El orden mide *lo rápido* que mejora el dibujo al acortar los segmentos. Un método de orden alto es un dibujante con mejor pulso: con pocos trazos ya clava la forma.

2. Milestone 1: tres métodos escritos a mano

Cada script de Milestone 1 lo hace todo en un único bucle: define el muelle, elige el paso y aplica el método.

2.1. `Problema_1.py`: Euler explícito

La idea

"Mira hacia dónde vas *ahora mismo* y camina recto en esa dirección durante Δt."

$$\boxed{\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\, \mathbf{F}(\mathbf{U}^n)} \qquad \text{nuevo estado} = \text{estado actual} + (\text{velocidad de cambio actual}) \times \Delta t.$$

Es lo más sencillo posible. El problema es que supone que el rumbo no cambia durante el tramo, y en un movimiento circular el rumbo cambia continuamente.

Analogía: el coche en la rotonda

Vas por una rotonda y, en vez de girar el volante poco a poco, lo bloqueas recto durante un segundo, lo corriges, lo vuelves a bloquear otro segundo, y así sucesivamente. Cada tramo recto te saca un poco hacia el exterior, así que acabas *saliéndote* de la rotonda en espiral.

Qué pasa en el oscilador

En un círculo la velocidad siempre es *perpendicular* al radio (tangente). Euler da un paso recto en esa dirección tangente, de longitud $\Delta t \cdot r$. Por **Pitágoras**, el radio viejo y el paso forman un triángulo rectángulo cuya hipotenusa es el radio nuevo (Fig. 6, izquierda):

$$r_{n+1}^2 = r_n^2 + (\Delta t\, r_n)^2 = (1 + \Delta t^2)\, r_n^2.$$

Cada paso aumenta la energía en un factor $1 + \Delta t^2$, sea cual sea Δt. Es un empujón fantasma: un columpio que sube cada vez más alto sin que nadie lo empuje. Con los valores del archivo ($\Delta t = 0.001$, 10 000 pasos, hasta $t = 10$) el radio final es $(1 + 10^{-6})^{5000} = 1.0050$, y el código da exactamente 1.0050125. Con un paso tan pequeño casi no se nota; con $\Delta t = 0.1$ el radio llegaría a 1.64 (Fig. 3).

El código

```python
import matplotlib.pyplot as plt
from numpy import array, zeros

# Modelo físico: oscilador armónico simple sin amortiguamiento.
# La variable U = [x, v] representa posición x y velocidad v.
# La ecuación del sistema es:
#   x' = v
#   v' = -x
# que corresponde a x'' + x = 0.
# Esto describe un movimiento circular/armónico en el plano (x, v).

def F(U):
    # Derivada del estado U = [x, v]
    return array([U[1], -U[0]])

# Número de pasos de tiempo
N = 10000
# Paso de integración (tiempo discreto)
At = 0.001
# Número de variables del estado
Nv = 2
```

```
22
23  # Matriz que guarda el estado en cada instante de tiempo:
24  # U[n,0] = x_n, U[n,1] = v_n
25  U = zeros((N + 1, Nv))
26
27  # Condición inicial: posición inicial x = 1, velocidad inicial v = 0
28  U[0, :] = array([1, 0])
29
30  # Integración explícita de Euler:
31  # U[n+1] = U[n] + Δt * F(U[n])
32  # Esta es la versión discreta de la ecuación de evolución del sistema.
33  for n in range(0, N):
34      U[n + 1, :] = U[n, :] + At * F(U[n, :])
35
36  # Graficamos la trayectoria en el plano (x, v), que para este sistema
37  # es una órbita circular alrededor del origen.
38  plt.plot(U[:, 0], U[:, 1])
39  plt.axis('equal')
40  plt.xlabel('Posición x')
41  plt.ylabel('Velocidad v')
42  plt.title('Órbita del oscilador armónico simple')
43  plt.show()
```

- **Líneas 12–14, la regla de cambio.** `F(U)` recibe la foto (x, v) y devuelve hacia dónde cambia, $(v, -x)$.
- **Líneas 17–21, los parámetros.** 10 000 pasos de $\Delta t = 0.001$ (`At`) y 2 variables por foto.
- **Línea 25, el cuaderno de bitácora.** Una tabla de $(N + 1)$ filas por 2 columnas: cada fila es un instante y guarda posición y velocidad.
- **Línea 28, el punto de partida.** Columpio apartado ($x = 1$) y quieto ($v = 0$).
- **Líneas 33–34, el método.** Para cada instante: siguiente fila = fila actual + `At` × `F`(fila actual). Es la fórmula del recuadro, tal cual.
- **Líneas 38–43, el dibujo.** Se pinta la velocidad frente a la posición (el plano de fases). `axis('equal')` hace que un círculo se vea redondo y no ovalado.

2.2. `Problema_2.py`: Runge–Kutta de orden 4 (RK4)

La idea

Euler solo mira la dirección al principio del tramo. RK4 **explora antes de decidir**: tantea la dirección en varios puntos del tramo y luego da el paso con una media ponderada.

Analogía: el excursionista en la niebla

Un excursionista en la niebla no ve el camino. Antes de avanzar, manda a un explorador:

1. $\mathbf{k}_1$: "¿Hacia dónde apunta el camino aquí?"
2. $\mathbf{k}_2$: el explorador avanza medio tramo siguiendo $\mathbf{k}_1$ y mira hacia dónde apunta el camino allí.
3. $\mathbf{k}_3$: vuelve a avanzar medio tramo, esta vez siguiendo $\mathbf{k}_2$ (que es mejor información), y mira otra vez.
4. $\mathbf{k}_4$: avanza el tramo entero siguiendo $\mathbf{k}_3$ y mira al final.

Con estos cuatro informes, el excursionista da el paso de verdad usando la media $\frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$. Los informes de mitad de camino cuentan doble porque son los más representativos del tramo, igual que en la regla de Simpson.

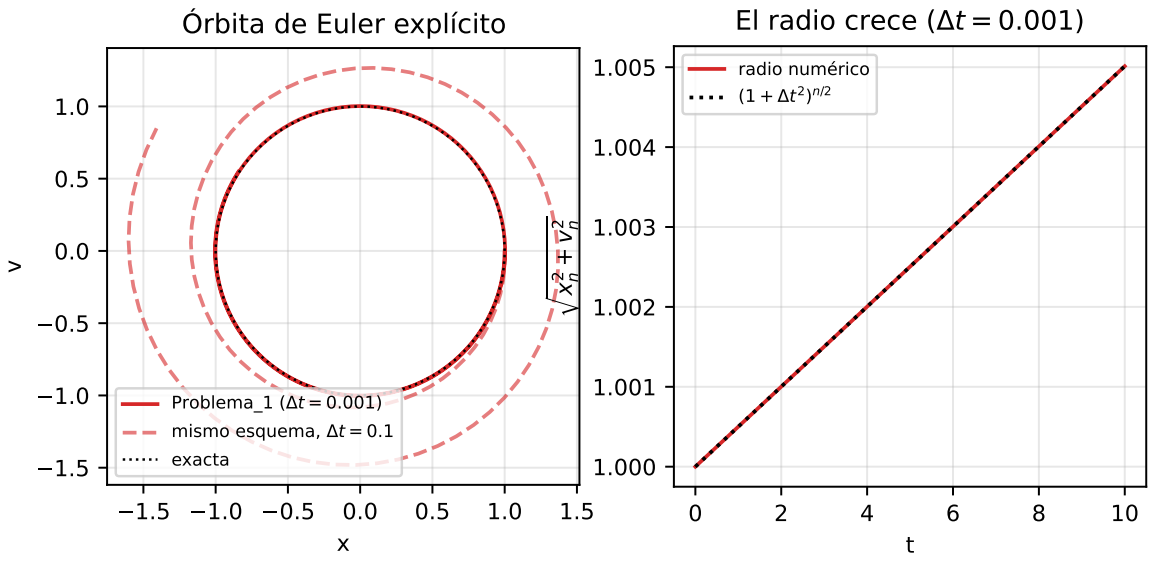


Figura 3: Izquierda: la órbita de `Problema_1.py` parece un círculo porque el paso es muy pequeño. La línea discontinua usa el mismo método con $\Delta t = 0.1$, y ahí la espiral se ve clarísima. Derecha: el radio crece exactamente como predice Pitágoras.

$$\mathbf{k}_1 = \mathbf{F}(\mathbf{U}^n), \quad \mathbf{k}_2 = \mathbf{F}(\mathbf{U}^n + \tfrac{\Delta t}{2}\mathbf{k}_1), \quad \mathbf{k}_3 = \mathbf{F}(\mathbf{U}^n + \tfrac{\Delta t}{2}\mathbf{k}_2), \quad \mathbf{k}_4 = \mathbf{F}(\mathbf{U}^n + \Delta t\,\mathbf{k}_3),$$

$$\boxed{\mathbf{U}^{n+1} = \mathbf{U}^n + \tfrac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)}$$

Cuesta 4 evaluaciones de $\mathbf{F}$ por paso, frente a 1 de Euler, pero es de **orden 4**: dividir el paso entre 2 divide el error entre 16.

Qué pasa en el oscilador

El giro exacto de un paso usa $\cos\Delta t$ y $\sin\Delta t$. Si se hacen las cuentas, RK4 hace el mismo giro pero con cos y sin sustituidos por sus primeros términos de Taylor:

$$\cos\Delta t \approx 1 - \tfrac{\Delta t^2}{2} + \tfrac{\Delta t^4}{24}, \qquad \sin\Delta t \approx \Delta t - \tfrac{\Delta t^3}{6}.$$

Es una **copia casi perfecta de la rotación**. Pierde una cantidad minúscula de energía por paso ($\rho^2 = 1 - \Delta t^6/72 + \dots$), un rozamiento fantasma casi imperceptible. El archivo usa $\Delta t = 0.1$ y 10^6 pasos, es decir, hasta $t = 100\,000$, unas **15 900 vueltas**. Aun así el radio solo baja a 0.99309, un 0.7 %. La teoría predice 0.993088232944 y el código da 0.993088232963.

El código

```
def F(U):
    # Derivada del estado U = [x, v]
    return array([U[1], -U[0]])

# Número total de pasos de tiempo
N = 1000000
# Tamaño del paso temporal
At = 0.1
# Número de componentes del vector de estado
Nv = 2

# Matriz que guarda la evolución del sistema:
# U[n,0] = x_n, U[n,1] = v_n
```

```
14 U = zeros((N + 1, Nv))
15
16 # Condición inicial: x(0)=1, v(0)=0
17 U[0, :] = array([1, 0])
18
19 # Integración numérica con el método de Runge-Kutta de cuarto orden (RK4).
20 # Este método aproxima mejor la solución que el método de Euler.
21 # En cada paso calculamos cuatro pendientes:
22 #   k1 = f(U_n)
23 #   k2 = f(U_n + At*k1/2)
24 #   k3 = f(U_n + At*k2/2)
25 #   k4 = f(U_n + At*k3)
26 # Luego se hace la combinación ponderada:
27 #   U_{n+1} = U_n + At*(k1 + 2*k2 + 2*k3 + k4)/6
28 for n in range(0, N):
29     k1 = F(U[n, :])
30     k2 = F(U[n, :] + At * k1 / 2)
31     k3 = F(U[n, :] + At * k2 / 2)
32     k4 = F(U[n, :] + At * k3)
33     U[n + 1, :] = U[n, :] + At * (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

- **Líneas 14–30:** el mismo muelle y el mismo cuaderno que antes, pero con $\Delta t = 0.1$ y un millón de pasos.
- **Líneas 42–45, los cuatro informes del explorador.** Por ejemplo, `U[n,:] + At*k1/2` es "el punto al que llega el explorador tras medio tramo siguiendo $\mathbf{k}_1$".
- **Línea 46:** el paso de verdad, con la media 1-2-2-1.
- **Coste:** cuatro millones de llamadas a `F`, unos 10 segundos en este ordenador. El dibujo superpone un millón de puntos sobre el mismo círculo.

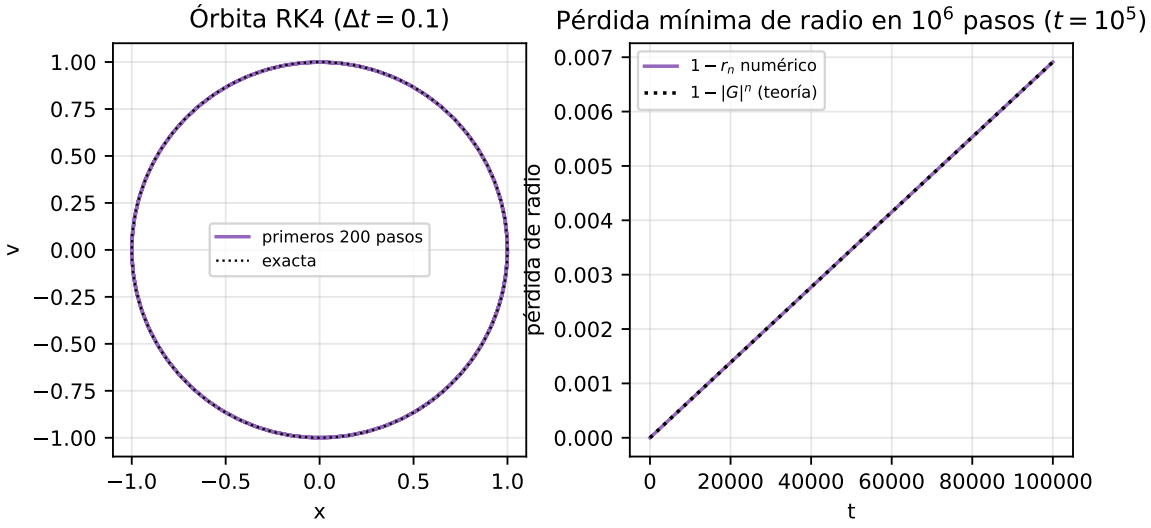


Figura 4: RK4 con $\Delta t = 0.1$. Izquierda: un círculo perfecto a simple vista. Derecha: la pérdida de radio en 10^6 pasos coincide con la teoría.

2.3. `Problema_2_2.py`: el método del trapecio (Crank–Nicolson) resuelto "a tanteo"

La idea

"Usa la *media* entre la dirección al salir y la dirección al llegar."

$$\boxed{\mathbf{U}^{n+1} = \mathbf{U}^n + \tfrac{\Delta t}{2}\Big(\mathbf{F}(\mathbf{U}^n) + \mathbf{F}(\mathbf{U}^{n+1})\Big)}$$

Hay una trampa: para saber la dirección al llegar necesitas saber a dónde llegas, que es justo lo que intentas calcular. El problema se muerde la cola. Los métodos así se llaman **implícitos**: en cada paso hay que *resolver una ecuación* en lugar de aplicar una fórmula directa. Este método se llama **trapecio** o **Crank–Nicolson (CN)** y es de **orden 2**.

Qué pasa en el oscilador

Euler daba el paso perpendicular al radio *de salida* y por eso se alejaba. CN lo da perpendicular al radio *del punto medio* del tramo, de modo que el paso es una **cuerda** del círculo: sale de la circunferencia y vuelve a caer exactamente sobre ella (Fig. 6, derecha).

$$\rho_{CN} = 1 \qquad \Longrightarrow \qquad \textbf{la energía se conserva exactamente.}$$

Su único defecto es que gira un pelín menos de lo debido: $\theta = 2\arctan(\Delta t/2) \approx \Delta t - \Delta t^3/12$. Es un **reloj de péndulo perfecto que atrasa un poco**. Con $\Delta t = 0.1$, tras $t = 10$ va atrasado 0.008 radianes, lo que da un error en $x(10)$ de -0.0045, justo lo que sale del código.

Cómo se resuelve la ecuación: adivinar y corregir

Llamamos $\mathbf{Y}$ al punto de llegada desconocido y medimos *cuánto falla* una propuesta con el **residuo**:

$$\mathbf{R}(\mathbf{Y}) = \mathbf{Y} - \mathbf{U}^n - \tfrac{\Delta t}{2}(\mathbf{F}(\mathbf{U}^n) + \mathbf{F}(\mathbf{Y})) \qquad (\mathbf{R} = \mathbf{0} \text{ quiere decir que } \mathbf{Y} \text{ es la respuesta correcta}).$$

El script empieza proponiendo $\mathbf{Y} = \mathbf{U}^n$ ("no me muevo") y repite la corrección $\mathbf{Y} \leftarrow \mathbf{Y} - \mathbf{R}(\mathbf{Y})$ hasta que el fallo es menor que 10^{-10}. Esto se llama **iteración de punto fijo**.

Analogía: ajustar la ducha

Abres el grifo, notas que el agua está fría (residuo), giras un poco, vuelves a notar, giras otra vez… Si cada ajuste deja el fallo en una fracción del anterior, acabas en la temperatura exacta. Aquí cada ajuste deja el fallo en el **5 %** del anterior (el factor es $\Delta t/2 = 0.05$). Partiendo de un fallo de 0.1 hacen falta **7 ajustes por paso**: $0.1 \to 0.005 \to 0.00025 \to \ldots$, que es justo la secuencia que imprime el programa (Fig. 5). Si el paso fuera grande ($\Delta t/2 \geq 1$), cada ajuste empeoraría las cosas y no convergería nunca. Esto motiva el método de Newton (Milestone 2).

Una nota sobre el nombre. Los comentarios del archivo lo llaman "punto medio implícito", pero la fórmula programada es la del trapecio. Para el oscilador da exactamente igual porque $\mathbf{F}$ es lineal (una recta), y en ese caso la media de las direcciones coincide con la dirección en el punto medio. Para problemas no lineales serían métodos distintos.

El código

```
for n in range(0, N):
    # Y es el valor aproximado del estado en el siguiente paso temporal.
    # Se inicializa con el valor actual para luego corregirlo iterativamente.
    Y = U[n, :]
```

```
5
6       # Mientras no se cumpla la ecuación implícita con la precisión pedida,
7       # seguimos ajustando Y.
8       # La expresión dentro de norm(...) es el residuo de la ecuación:
9       #   R = Y - U[n] - (At/2) * (F(U[n]) + F(Y))
10      # Cuando R ≈ 0, entonces Y satisface el método del punto medio implícito.
11      while norm(Y - U[n, :] - At / 2 * (F(U[n, :]) + F(Y))) > 1e-10:
12          # Calculamos el residuo residual para saber cuánto falta para cumplir la ecuación.
13          R = Y - U[n, :] - At / 2 * (F(U[n, :]) + F(Y))
14
15          # Corregimos Y restando el residuo. Esto mueve Y hacia la solución de la ecuación implí
    cita.
16          # Si R es positivo, disminuimos Y; si es negativo, aumentamos Y.
17          Y = Y - R
18
19          # Muestra la magnitud del error residual en cada iteración.
20          # Sirve para ver cómo converge la solución.
21          print(norm(R))
22          # input("Press Enter to continue...")
23
24      # Cuando el residuo es suficientemente pequeño, aceptamos Y como el siguiente estado.
25      U[n + 1, :] = Y
```

- **Línea 41:** primera propuesta, “me quedo donde estoy”.
- **Línea 48:** mientras el fallo $\|\mathbf{R}\|$ sea mayor que 10^{-10}, seguimos corrigiendo.
- **Líneas 50–54:** calcular el fallo y restárselo a la propuesta. (El fallo se calcula dos veces por vuelta, en la condición y dentro; guardándolo una vez sería más rápido.)
- **Línea 58:** imprime el tamaño del fallo para ver cómo baja: 7 números por paso, 700 líneas en total.
- **Línea 62:** se acepta la propuesta como siguiente fila del cuaderno.

El radio final es 0.99999999 y no exactamente 1 porque se para de corregir cuando el fallo es “suficientemente pequeño”, no cero. Es como dejar la ducha a 0.0001 °C de la temperatura perfecta.

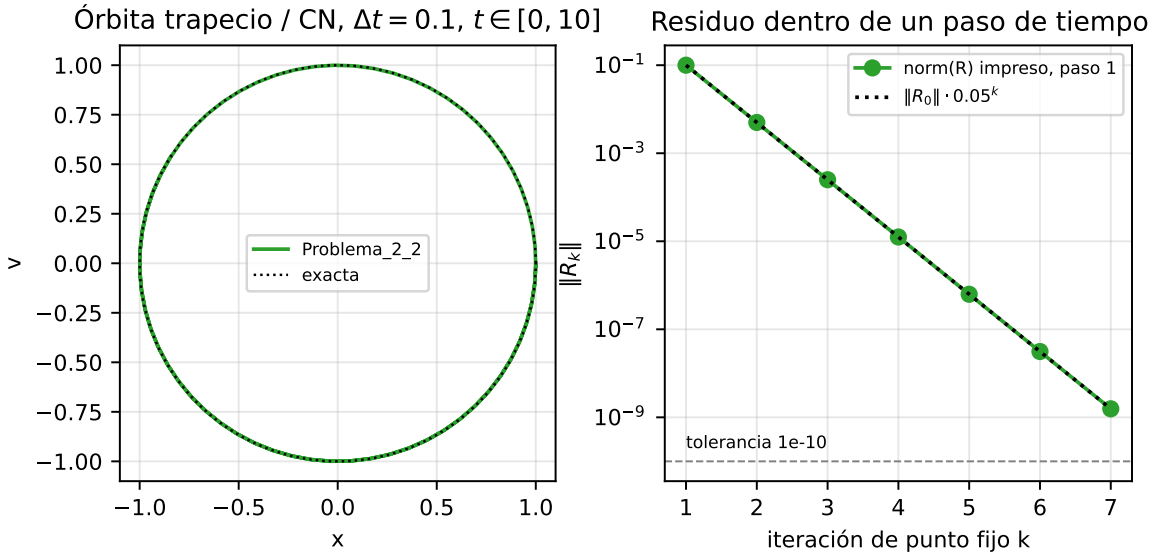


Figura 5: Izquierda: CN se queda en el círculo. Derecha: el fallo dentro de un paso; cada ajuste lo multiplica por 0.05.

3. Milestone 2: ordenar el código en piezas intercambiables

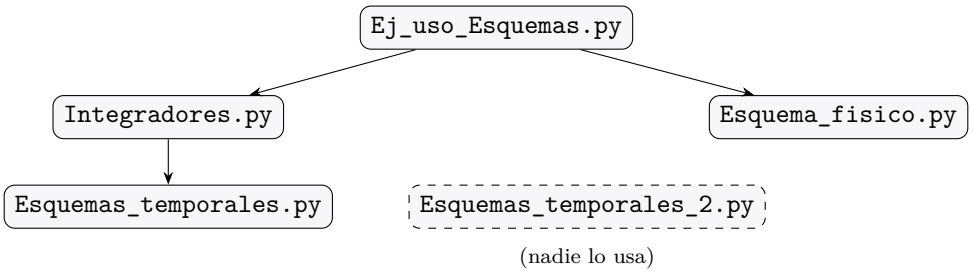
En Milestone 1 cada script mezclaba todo. La idea de Milestone 2 es que *cualquier* método de un paso hace lo mismo:

$$(\text{estado actual, tamaño del paso, instante, regla de cambio } \mathbf{F}) \longmapsto \text{estado siguiente.}$$

Si todos los métodos reciben y devuelven lo mismo, se pueden intercambiar como piezas.

Analogía: la videoconsola

- **El juego** es la física, **F** (`Esquema_fisico.py`). Hoy es un muelle; mañana podría ser un planeta.
- **Los mandos** son los métodos EE, EI, CN, RK4 (`Esquemas_temporales.py`). Todos tienen el mismo conector (`U, At, t, F`).
- **La consola** es el bucle de tiempo (`Integradores.py`). Le da igual qué juego o qué mando le pongas: los hace funcionar.
- **La partida** es el experimento (`Ej_uso_Esquemas.py`).



3.1. `Esquema_fisico.py`: la física

La idea

Es la regla de cambio del muelle, $\mathbf{F}(\mathbf{U}, t) = (v, -x)$, que ahora también recibe el tiempo t. El muelle no lo usa, porque se comporta igual a cualquier hora, pero así el mismo código serviría para un columpio al que alguien empuja periódicamente, donde la fuerza sí depende de la hora.

El código

```python
from array import array

def oscilador(U:array, t:float):
    """Ecuación del oscilador armónico simple.
    U = [x, v], entonces el sistema se define como:
        dx/dt = v
        dv/dt = -x
    Esto corresponde a la ecuación diferencial x'' + x = 0.
    """
    # Estado del sistema: U[0] = posición, U[1] = velocidad.
    return array([U[1], -U[0]])
```

Error encontrado en el código

Línea 1: `from array import array` no trae la `array` de NumPy, sino una homónima de la biblioteca estándar de Python que funciona de otra manera (exige un código de tipo, como `array('d', [...])`). Es como pedir "el martillo" y que te den otro martillo con el mismo nombre pero distinto mango. Al llamar a la función salta `TypeError: array() argument 1 must be a unicode character, not list` y el ejemplo de Milestone 2 no llega a ejecutarse. **Solución:** `from numpy import array`. Ya está corregido en Milestone 3.

3.2. `Esquemas_temporales.py`: los cuatro métodos como funciones

La idea

Se empaquetan EE, CN y RK4 (ya vistos) y aparece uno nuevo, **Euler implícito (EI)**:

$$\boxed{\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\, \mathbf{F}(\mathbf{U}^{n+1})} \qquad \text{"camina en la dirección que tendrás } \textit{al llegar}\text{"}.$$

Es el hermano "pesimista" de Euler explícito. También es implícito (se muerde la cola) y de **orden 1**.

Qué pasa en el oscilador: los tres hermanos y Pitágoras

EI da el paso perpendicular al radio *de llegada*. Ahora el radio viejo es la hipotenusa del triángulo, así que el nuevo es más corto (Fig. 6, centro):

$$r_n^2 = r_{n+1}^2 + (\Delta t\, r_{n+1})^2 \quad \Longrightarrow \quad r_{n+1}^2 = \frac{r_n^2}{1+\Delta t^2}.$$

Es un **rozamiento fantasma**: el columpio se va parando aunque en la realidad no hay rozamiento. Con $\Delta t = 0.1$ hasta $t = 10$: EE acaba con radio 1.645 y EI con 0.608. Son simétricos: uno multiplica y el otro divide por lo mismo. CN, que es la media de los dos, se queda exactamente en 1.

EE: paso $\perp$ radio de *salida*	$\Rightarrow$ crece	empujón fantasma
EI: paso $\perp$ radio de *llegada*	$\Rightarrow$ decrece	rozamiento fantasma
CN: paso $\perp$ radio *del medio*	$\Rightarrow$ cuerda, se conserva	reloj que atrasa un poco

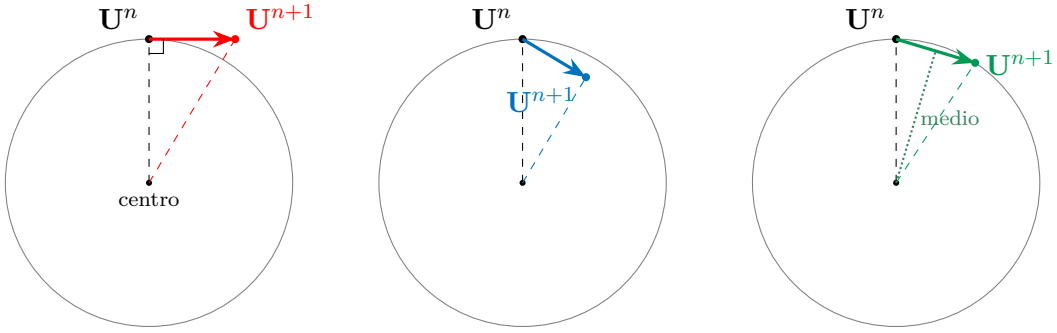
¿Por qué interesa EI si pierde energía? Porque es *muy estable*: nunca explota, dé el paso que dé. En problemas que físicamente tienden a calmarse (calor que se difunde, reacciones químicas rápidas) esa robustez es oro.

El código

```
'''Esquemas temporales para la integración de ecuaciones diferenciales ordinarias (EDO)
    -Euler Explícito- EE
    -Euler Implícito- EI
    -Crank-Nicolson- CN
    -Runge-Kutta de 4to orden- RK4
'''
import numpy as np
from numpy import array, zeros
from numpy.linalg import norm
from typing import Callable

def EE(U:array ,At:float,t:float, F:Callable[[array,float], array]):
    return U+ At * F(U,t)

def EI(U:array, At:float, t:float, F:Callable[[array,float], array]):
```



EE: termina *fuera* **EI**: termina *dentro* **CN**: termina *sobre* el círculo

Figura 6: Un paso de cada método, exagerado ($\Delta t = 0.6$). El movimiento va en sentido horario. EE avanza por la tangente del punto de salida y Pitágoras alarga el radio. EI avanza por la tangente del punto de llegada y el radio se acorta. CN avanza perpendicular al radio del punto medio (punteado), es decir, por una cuerda, y el radio se mantiene.

```
16     # Método de Euler implícito para un solo paso: U(t+At) = U(t) + F(U(t+At), t+At)
17     Y = U + At* F(U, t)
18     while norm(Y - U - At* F(Y,t+At)) > 1e-10:
19        R = Y - U - At * F(Y,t+At)
20        Y = Y - R
21     return Y
22
23 def CN(U:array, At:float, t:float, F:Callable[[array,float], array]):
24     # Método de Crank-Nicolson de un solo paso: U(t+At) = U(t) + At/2 * {F(U(t), t) + F(U(t+At), t
       +At)}.
25     Y = U + At * F(U, t)
26     while norm(Y - U - At/2 * (F(U, t) + F(Y, t + At))) > 1e-10:
27         R = Y - U - At/2 * (F(U, t) + F(Y, t + At))
28         Y = Y - R
29     return Y
30
31 def RK4(U:array, At:float, t:float, F:Callable[[array,float], array]):
32     # Esquema básico de Runge-Kutta de cuarto orden.
33     k1 = F(U,t)
34     k2 = F(U, t + At * k1 / 2)
35     k3 = F(U, t + At * k2 / 2)
36     k4 = F(U, t + At * k3)
37     return U + At * (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

- **Todas las funciones** tienen el mismo conector `(U, At, t, F)` y devuelven el estado siguiente. Por eso son intercambiables.
- **Línea 13 (EE):** una sola línea, la fórmula tal cual.
- **Líneas 17–21 (EI):** primera propuesta, "lo que diría Euler explícito" (más lista que "no me muevo"), y luego ajustar la ducha hasta que el fallo $\mathbf{R} = \mathbf{Y} - \mathbf{U} - \Delta t\mathbf{F}(\mathbf{Y})$ sea diminuto. Cada ajuste deja el fallo en el 10 % del anterior (factor $\Delta t = 0.1$). (Al comentario de la línea 16 le falta el `At`; el código está bien.)
- **Líneas 25–29 (CN):** igual, pero con el fallo del trapecio.

Error encontrado en el código: el RK4 de Milestone 2 es en realidad Euler

En las líneas 34–36 el explorador *no se mueve*: `F(U, t + At*k1/2)` suma el avance al *reloj* en lugar de a la *posición*. Lo correcto es

`k2 = F(U + At*k1/2, t + At/2).`

Como al muelle le da igual la hora, el explorador informa cuatro veces exactamente lo mismo ($\mathbf{k}_1 = \mathbf{k}_2 = \mathbf{k}_3 = \mathbf{k}_4$). La media 1-2-2-1 de cuatro cosas iguales es esa misma cosa, así que el resultado es **Euler explícito disfrazado**. Ejecutándolo, la diferencia entre "RK4" y EE es de 4×10^{-16}, puro redondeo del ordenador. Corregido en Milestone 3.

3.3. `Esquemas_temporales_2.py`: el método de Newton

La idea

Ajustar la ducha a ciegas funciona, pero lentamente, y si el paso es grande no funciona. Newton es más listo: **mide la sensibilidad** y salta directamente a la solución.

Analogía: la ducha con manual

Si sabes que "cada vuelta del grifo sube el agua 5 °C" (la sensibilidad) y el agua está 10 °C fría, das dos vueltas de golpe y aciertas. Esa sensibilidad es la **matriz jacobiana**. Cuando la relación grifo–temperatura es una recta (problema lineal, como el muelle), **aciertas al primer intento**. Si es una curva, aciertas casi del todo y en 2 o 3 intentos lo clavas. Cada intento duplica más o menos el número de cifras correctas.

En fórmulas: buscamos el cero del fallo $\mathbf{R}(\mathbf{Y}) = \mathbf{Y} - \mathbf{U}^n - \Delta t\, \mathbf{F}(\mathbf{Y})$. La sensibilidad del fallo es $J = I - \Delta t\, \partial\mathbf{F}/\partial\mathbf{U}$, y el salto se obtiene resolviendo

$$J\,\boldsymbol{\delta} = \mathbf{R}(\mathbf{Y}), \qquad \mathbf{Y}_{\text{nuevo}} = \mathbf{Y} - \boldsymbol{\delta}.$$

¿Cómo se calcula la sensibilidad? "Tocando cada mando un poquito y viendo cómo responde": se mueve un poquito ($h = 10^{-8}$) la posición y se mira cuánto cambia $\mathbf{F}$; luego se repite con la velocidad. Cada prueba rellena una columna de la jacobiana.

El truco para CN. CN se *disfraza* de Euler implícito. Se define una regla de cambio auxiliar $G(\mathbf{Y}) = \frac{1}{2}(\mathbf{F}(\mathbf{U}^n) + \mathbf{F}(\mathbf{Y}))$, la media de salida y llegada, y se le pide a Newton que resuelva "Euler implícito con G". Las cuentas salen idénticas a CN, y así una sola función de Newton sirve para los dos métodos.

Qué pasa en el oscilador

El muelle es lineal (fuerza $= -x$, una recta), así que Newton acierta **en un solo salto**. Comprobado: un único cálculo de jacobiana por paso, y el resultado coincide con resolver el sistema 2×2 a mano. El ajuste a ciegas necesitaba 7–10 intentos.

El código

```
1  def EI(U:array, At:float, t:float, F:Callable[[array,float], array]):
2      # Método de Euler implícito para un solo paso: U(t+At) = U(t) + At * F(U(t+At), t+At)
3      return My_Newton(U, At, t, F)
4
5  def CN(U:array, At:float, t:float, F:Callable[[array,float], array]):
6      # Método de Crank-Nicolson de un solo paso: U(t+At) = U(t) + At/2 * {F(U(t), t) + F(U(t+At), t
       +At)}.
7      def G(X:array, time:float):
```

```
8           return (F(U, t) + F(X, time)) / 2
9
10      return My_Newton(U, At, t, G)
11
12  def RK4(U:array, At:float, t:float, F:Callable[[array,float], array]):
13      # Esquema básico de Runge-Kutta de cuarto orden.
14      k1 = F(U,t)
15      k2 = F(U, t + At * k1 / 2)
16      k3 = F(U, t + At * k2 / 2)
17      k4 = F(U, t + At * k3)
18      return U + At * (k1 + 2 * k2 + 2 * k3 + k4) / 6
19
20  def My_Newton(U:array, At:float, t:float, F:Callable[[array,float], array],
21                tol:float=1e-10, max_iter:int=50):
22      # Método de Newton para resolver ecuaciones no lineales.
23      Y = U + At * F(U, t)
24      for _ in range(max_iter):
25          R = Y - U - At * F(Y, t + At)
26          if norm(R) <= tol:
27              return Y
28          J = np.eye(len(U)) - At * My_Jacobian(F, Y, t + At)
29          Y = Y - np.linalg.solve(J, R)
30
31      raise RuntimeError(f"Newton no convergió en {max_iter} iteraciones.")
32
33  def My_Gauss_Seidel(F:Callable[[array,float], array], U:array, At:float, t:float):
34      # Método de Gauss-Seidel para resolver ecuaciones no lineales.
35      Y = U + At * F(U, t)
36      while norm(Y_new - Y) < 1e-10:
37          Y_new = U + At * F(Y, t + At)
38      Y = Y_new
39      return Y
40
41  def My_Jacobian(F:Callable[[array,float], array], U:array, t:float):
42      # Aproximación de la matriz jacobiana de F en el punto U.
43      n = len(U)
44      J = np.zeros((n, n))
45      h = 1e-8
46      for i in range(n):
47          U_plus = U.copy()
48          U_plus[i] += h
49          J[:, i] = (F(U_plus, t) - F(U, t)) / h
50      return J
```

- **Línea 20 (EI):** se lo encarga a Newton.
- **Líneas 24–27 (CN):** define la regla auxiliar `G` (que "recuerda" el punto de salida `U`) y se la pasa al mismo Newton.
- **Línea 35:** el RK4 tiene el mismo error que en el archivo anterior.
- **Líneas 40–48 (Newton):** propuesta inicial con Euler, y como mucho 50 intentos. En cada uno: calcular el fallo; si es pequeño, terminar; si no, calcular la sensibilidad `J` y resolver el sistema con `np.linalg.solve` (más rápido y preciso que invertir la matriz). Si en 50 intentos no lo consigue, avisa con un error en vez de devolver basura.
- **Líneas 58–67 (jacobiana):** para cada variable, copiar el estado (`U.copy()`, para no estropear el original), empujarla un poquito y anotar cómo cambia `F`.

Error encontrado en el código: `My_Gauss_Seidel` (líneas 50–56)

Pretende ser el ajuste a ciegas, pero usa `Y_new` antes de haberlo calculado (`UnboundLocalError`), la comparación está al revés (`<` en lugar de `>`), y la actualización `Y = Y_new` está fuera del bucle. Nadie la llama. Además, `Integradores.py` no importa este archivo, así que en Milestone 2 Newton aún no se usa.

3.4. `Integradores.py`: la consola, o el bucle de tiempo

La idea

Divide el intervalo de tiempo $[t_0, t_f]$ en N trozos iguales, como las marcas de una regla, y aplica el método elegido una y otra vez:

$$\mathbf{U}^0 = \mathbf{U}_0, \qquad \mathbf{U}^{n+1} = \text{método}(\mathbf{U}^n,\ \Delta t,\ t_n,\ \mathbf{F}), \quad n = 0, 1, \ldots, N-1.$$

Es el bucle de Milestone 1, pero escrito una sola vez para cualquier método y cualquier física. En el ejemplo: de $t = 0$ a $t = 10$ con $N = 100$, así que $\Delta t = 0.1$.

El código

```
1  import numpy as np
2  from typing import Callable
3
4  def partition(t0:float, tf:float, N:float):
5      # Crea una partición del intervalo [t0, tf] en N subintervalos equiespaciados.
6      return np.array([t0 + i * (tf - t0) / N for i in range(N + 1)])
7
8  def integrar_Cauchy(
9      esquema: str,
10     U0: np.ndarray,
11     N_t: int, # Numero de pasos de tiempo
12     t0: float,
13     tf: float,
14     F: Callable[[np.ndarray, float], np.ndarray],
15 ):
16     if esquema == "EE":
17         from Esquemas_temporales import EE as metodo
18     elif esquema == "EI":
19         from Esquemas_temporales import EI as metodo
20     elif esquema == "CN":
21         from Esquemas_temporales import CN as metodo
22     elif esquema == "RK4":
23         from Esquemas_temporales import RK4 as metodo
24
25     U = np.zeros((N_t + 1, len(U0)))
26     U[0, :] = U0
27     t = partition(t0, tf, N_t)
28     for n in range(0, N_t):
29         U[n + 1, :] = metodo(U[n, :], t[n + 1] - t[n], t[n], F)
30     return U
```

- **Líneas 4–6, `partition`:** las $N + 1$ marcas de tiempo. Equivale a `np.linspace(t0, tf, N+1)`. (La pista de tipo `N: float` debería ser `int`.)
- **Líneas 16–23:** elige el método según su nombre. Si el nombre no existe, `metodo` queda sin definir y el error salta más adelante, de forma poco clara.
- **Líneas 25–29:** crea el cuaderno, apunta el estado inicial y repite el método. El paso se calcula como `t[n+1]-t[n]`, así que funcionaría aunque las marcas no fueran equidistantes.

3.5. `Ej_uso_Esquemas.py`: la carrera de los cuatro métodos

La idea

Se lanzan los cuatro métodos desde el mismo punto, con el mismo paso, y se comparan con Euler explícito.

Analogía: juzgar una carrera

Comparar contra EE es como juzgar a los corredores por lo lejos que quedan *del peor corredor*, en vez de por lo lejos que quedan *de la meta*. Mide "cuánto se diferencian de EE", no su error real. Para el muelle conocemos la meta, $(\cos t, -\sin t)$. Cuando no se conoce, hay que *estimar* el error, y eso es justo lo que hace Milestone 3.

El código

```
1  N_t = 100 #Numero de pasos de tiempo
2  Nv = 2 #dimesiones del problema (x,v)
3  t_i=0 #tiempo inicial
4  t_f=10 #tiempo final
5  U_1 = zeros((N_t + 1, Nv))
6  U_1[0, :] = array([1, 0])
7  U_2 = zeros((N_t + 1, Nv))
8  U_2[0, :] = array([1, 0])
9  U_3 = zeros((N_t     + 1, Nv))
10 U_3[0, :] = array([1, 0])
11 U_4 = zeros((N_t + 1, Nv))
12 U_4[0, :] = array([1, 0])
13
14 U_1 = integrar_Cauchy("EE", U_1[0, :] , N_t, t_i, t_f, F)
15 U_2 = integrar_Cauchy("RK4", U_2[0, :] , N_t, t_i, t_f, F)
16 U_3 = integrar_Cauchy("EI", U_3[0, :] , N_t, t_i, t_f, F)
17 U_4 = integrar_Cauchy("CN", U_4[0, :] , N_t, t_i, t_f, F)
18
19 ERR_RK4 = abs(U_1 - U_2)
20 ERR_EI = abs(U_1 - U_3)
21 ERR_CN = abs(U_1 - U_4)
```

- **Líneas 18–25:** se crean cuatro cuadernos pero solo se usa su primera fila, porque `integrar_Cauchy` crea el suyo propio. Bastaría con `U0 = array([1,0])`.
- **Líneas 27–30:** la ventaja del diseño por piezas: misma física y mismo punto de partida, con cuatro mandos distintos.
- **Líneas 32–34:** diferencias con EE, que luego se dibujan.

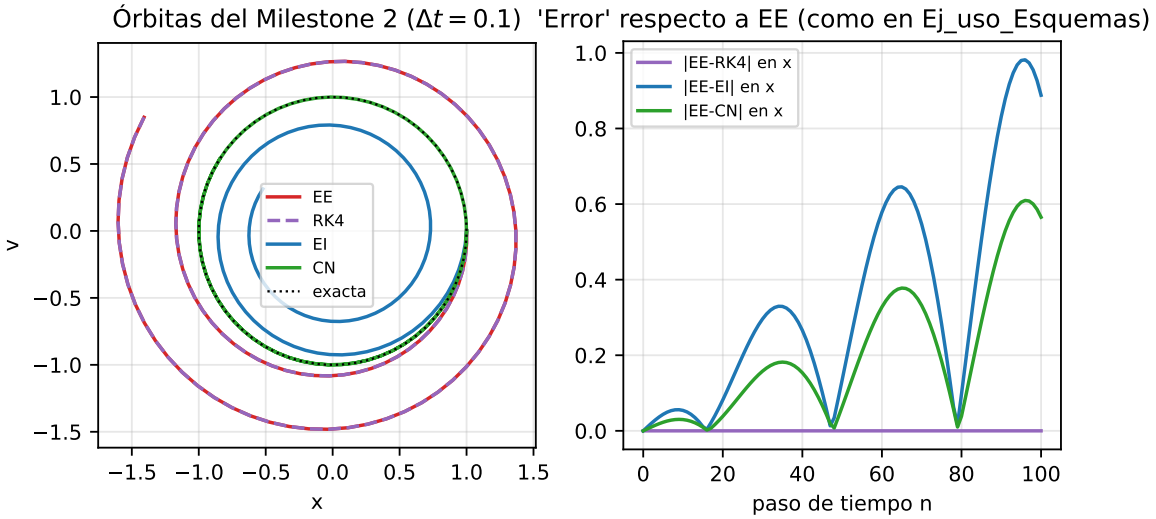
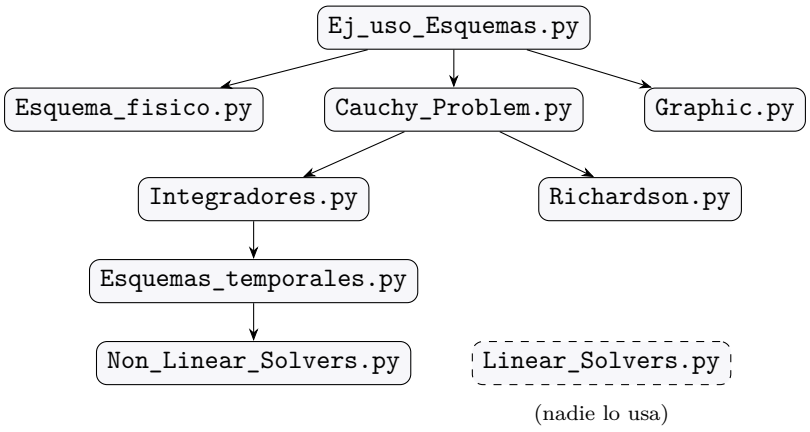


Figura 7: Milestone 2 ejecutado con una **F** de NumPy, porque el archivo original falla. EE sale hacia fuera (1.645), EI cae hacia dentro (0.608) y CN se queda en el círculo. "RK4" está exactamente encima de EE y su diferencia con EE es cero: es el error del explorador que no se mueve.

4. Milestone 3: ¿cuánto me estoy equivocando?

Milestone 3 corrige los errores anteriores, separa los "resolvedores" en sus propios archivos y añade la gran novedad: **estimar el error y el orden de un método sin conocer la solución exacta**. El muelle sirve de examen, porque con él sí conocemos la respuesta y podemos comprobar si la estimación acierta.



4.1. `Esquema_fisico.py`: la física, ya corregida

Es la misma regla del muelle, $\mathbf{F} = (v, -x)$.

```python
import numpy as np


def oscilador(U: np.ndarray, t: float) -> np.ndarray:
    """Ecuación del oscilador armónico simple.
    U = [x, v], entonces el sistema se define como:
        dx/dt = v
        dv/dt = -x
    Esto corresponde a la ecuación diferencial x'' + x = 0.
    """
    # Estado del sistema: U[0] = posición, U[1] = velocidad.
    return np.array([U[1], -U[0]])
```

Ahora usa `np.array` (el martillo correcto) y las pistas de tipo (`np.ndarray`) dejan claro qué recibe y qué devuelve. Cualquier otra física (un péndulo, un planeta…) tendría que seguir este mismo molde.

4.2. `Non_Linear_Solvers.py`: Newton en su propio archivo

Es el Newton y la jacobiana de `Esquemas_temporales_2.py` (la "ducha con manual"), trasladados aquí para que los métodos implícitos los usen. Con el muelle sigue acertando en un solo salto.

```python
import numpy as np
from numpy.linalg import norm
from numpy import array
from typing import Callable

def My_Newton(U:array, At:float, t:float, F:Callable[[array,float], array],
              tol:float=1e-10, max_iter:int=50):
    # Método de Newton para resolver ecuaciones no lineales.
    Y = U + At * F(U, t)
    for _ in range(max_iter):
        R = Y - U - At * F(Y, t + At)
        if norm(R) <= tol:
            return Y
        J = np.eye(len(U)) - At * My_Jacobian(F, Y, t + At)
        Y = Y - np.linalg.solve(J, R)

    raise RuntimeError(f"Newton no convergió en {max_iter} iteraciones.")

def My_Jacobian(F:Callable[[array,float], array], U:array, t:float):
    # Aproximación de la matriz jacobiana de F en el punto U.
    n = len(U)
    J = np.zeros((n, n))
    h = 1e-8
    for i in range(n):
        U_plus = U.copy()
        U_plus[i] += h
        J[:, i] = (F(U_plus, t) - F(U, t)) / h
    return J
```

Un comentario de diseño: `My_Newton` solo sabe resolver fallos con la forma concreta "$\mathbf{Y} - \mathbf{U} - \Delta t\mathbf{F}$". Sería más reutilizable un `Newton(R, Y0)` genérico que encuentre el cero de *cualquier* función `R`, y que cada método construyera su propio fallo.

4.3. `Linear_Solvers.py`: el ajuste a ciegas, ya corregido

Es la "ducha sin manual" para Euler implícito: proponer, recalcular $\mathbf{Y} \leftarrow \mathbf{U} + \Delta t\mathbf{F}(\mathbf{Y})$, y parar cuando dos propuestas seguidas casi coinciden. Con el muelle, cada ajuste deja el error en el 10 %.

```python
import numpy as np
from numpy.linalg import norm
from numpy import array
from typing import Callable

def My_Gauss_Seidel(F:Callable[[array,float], array], U:array, At:float, t:float):
    # Método de Gauss-Seidel para resolver ecuaciones no lineales.
    Y = U + At * F(U, t)
    while True:
        Y_new = U + At * F(Y, t + At)
        if norm(Y_new - Y) < 1e-10:
            return Y_new
        Y = Y_new
```

Los fallos de Milestone 2 están arreglados: primero calcula `Y_new` y luego compara. Dos matices de nombre: el método de *Gauss–Seidel* de verdad sirve para sistemas lineales y va actualizando las incógnitas una a una, mientras que esto es una iteración de punto fijo; y el archivo se llama "Linear" pero resuelve una ecuación que en general no es lineal. De todas formas nadie lo usa.

4.4. `Esquemas_temporales.py`: los cuatro métodos, ahora bien

```
'''Esquemas temporales para la integración de ecuaciones diferenciales ordinarias (EDO)
    -Euler Explícito- EE
    -Euler Implícito- EI
    -Crank-Nicolson- CN
    -Runge-Kutta de 4to orden- RK4

    También se calculan las funciones de apoyo de Newton, Gauss-Seidel y Jacobiano para los mé
    todos implícitos.
'''
import numpy as np
import Non_Linear_Solvers
from numpy import array
from numpy.linalg import norm
from typing import Callable


def EE(U:array, At:float, t:float, F:Callable[[array,float], array]):
    return U + At * F(U, t)


def EI(U:array, At:float, t:float, F:Callable[[array,float], array]):
    # Método de Euler implícito mediante Newton.
    return Non_Linear_Solvers.My_Newton(U, At, t, F)


def CN(U:array, At:float, t:float, F:Callable[[array,float], array]):
    # Método de Crank-Nicolson mediante Newton.
    def G(X:array, time:float):
        return (F(U, t) + F(X, time)) / 2
    return Non_Linear_Solvers.My_Newton(U, At, t, G)


def RK4(U:array, At:float, t:float, F:Callable[[array,float], array]):
    # Esquema básico de Runge-Kutta de cuarto orden.
    k1 = F(U, t)
    k2 = F(U + At * k1 / 2, t + At / 2)
    k3 = F(U + At * k2 / 2, t + At / 2)
    k4 = F(U + At * k3, t + At)
    return U + At * (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

- **Líneas 20–29:** EI y CN usan Newton. CN con el disfraz de la regla auxiliar `G`.
- **Líneas 34–38:** RK4 **corregido**: ahora el explorador sí se mueve (`U + At*k1/2`) y el reloj avanza lo que toca (`t + At/2`).

Qué pasa en el oscilador

Un paso de RK4 desde $(1, 0)$ con $\Delta t = 0.1$ da $(0.99500417, -0.09983333)$, y lo exacto es $(0.99500417, -0.09983342)$. La diferencia, 8×10^{-8}, es justo el primer término que RK4 "se deja" del desarrollo del seno, $\Delta t^5/120$. El explorador ya hace su trabajo.

4.5. `Integradores.py`: la consola, con carta de restaurante

```python
import numpy as np
from typing import Callable

from Esquemas_temporales import CN, EE, EI, RK4


def partition(t0: float, tf: float, N: float):
    # Crea una partición del intervalo [t0, tf] en N subintervalos equiespaciados.
    return np.array([t0 + i * (tf - t0) / N for i in range(N + 1)])


def integrar_Cauchy(
    esquema: str,
    U0: np.ndarray,
    N_t: int,
    t0: float,
    tf: float,
    F: Callable[[np.ndarray, float], np.ndarray],
):
    metodos = {"EE": EE, "EI": EI, "CN": CN, "RK4": RK4}
    U = np.zeros((N_t + 1, len(U0)))
    U[0, :] = U0
    t = partition(t0, tf, N_t)
    metodo = metodos[esquema]
    for n in range(N_t):
        U[n + 1, :] = metodo(U[n, :], t[n + 1] - t[n], t[n], F)
    return U
```

Hace lo mismo que en Milestone 2. La mejora está en la línea 20: un **diccionario** funciona como la carta de un restaurante, donde el nombre del plato (`"RK4"`) te lleva al plato (la función `RK4`). Si pides algo que no está en la carta, Python avisa claramente (`KeyError`).

4.6. `Richardson.py`: estimar el error sin conocer la respuesta

La idea

Analogía: medir con dos reglas

Mides una mesa con una regla grande y tosca y después con una el doble de fina. No sabes la medida exacta, pero **lo que cambia el resultado al afinar la regla te dice cuánto te equivocabas**. Si sabes además a qué ritmo mejora la regla al afinarla (el orden), puedes calcular cuánto error queda todavía.

Se resuelve el problema dos veces: con paso Δt ("grueso") y con paso $\Delta t/2$ ("fino"), y se comparan en los mismos instantes. Para un método de orden q, el error se divide entre 2^q al pasar de grueso a fino. Con un poco de álgebra:

$$\boxed{\text{error del fino} \approx \frac{\text{grueso} - \text{fino}}{2^q - 1}}, \qquad \text{error del grueso} \approx 2^q \times \text{error del fino}.$$

Por ejemplo, en un método de orden 1 el error del fino es igual a la diferencia entre ambos, y en uno de orden 4 es la diferencia entre 15. Lo bonito es que **no hace falta la solución exacta**. De regalo, si al fino le restas su error estimado obtienes una solución aún mejor; eso es la *extrapolación de Richardson*.

El código

```python
import numpy as np

def error_richardson(
```

```
    coarse: np.ndarray,
    fine: np.ndarray,
    order: float,
) -> np.ndarray:
    """
    Estima el error de la aproximación fina mediante extrapolación de Richardson.
    'coarse' y 'fine' deben representar la misma magnitud en los mismos tiempos,
    calculada con pasos h y h/n, respectivamente.
    """
    return (coarse - fine) / (2**order - 1)
```

La línea 13 es la fórmula del recuadro, aplicada de una vez a toda la tabla (todos los instantes, posición y velocidad). Como dice el comentario, devuelve el error del *fino*.

4.7. `Cauchy_Problem.py`: medir el orden en la práctica

La idea

Para usar Richardson hace falta el orden q, y también se puede *medir*. Se resuelve con N, $2N$ y $4N$ pasos y se mira cuánto se reduce la diferencia al afinar:

$$\frac{\text{diferencia}(N,\,2N)}{\text{diferencia}(2N,\,4N)} \approx 2^q \qquad \Longrightarrow \qquad q \approx \log_2(\text{ese cociente}).$$

Si al afinar las diferencias se reducen a la mitad, el orden es 1; si se reducen a la cuarta parte, orden 2; si a la dieciseisava parte, orden 4.

Qué pasa en el oscilador

Esto es lo que imprime el programa para el muelle ($N = 100$, $\Delta t = 0.1$, hasta $t = 10$), y lo que sale con $N = 1000$:

	EE	EI	CN	RK4
orden teórico	1	1	2	4
orden medido, $N = 100$	1.21	0.80	2.00	4.00
orden medido, $N = 1000$	1.02	0.98	2.00	4.00

¿Por qué EE y EI no dan 1 con $N = 100$? La regla "el error se divide entre 2^q" solo se cumple cuando el error ya es pequeño. Con $\Delta t = 0.1$, en $t = 10$ la espiral de EE ya ha crecido un 64 %. Ahí las aproximaciones "para cosas pequeñas" fallan, como cuando $e^{0.5}$ ya no se parece a $1 + 0.5$. Con pasos más pequeños ($N = 1000$) el orden medido se acerca a 1. CN y RK4 cometen errores tan pequeños que ya están en el régimen bueno.

El código

```
import numpy as np
import Integradores
import Richardson
from typing import Callable

def sol_error_cauchy(
    esquema: str,
    U0: np.ndarray,
    N_t: int,
    t0: float,
    tf: float,
    F: Callable[[np.ndarray, float], np.ndarray],
):
    coarse = Integradores.integrar_Cauchy(esquema, U0, N_t, t0, tf, F)
```

```
    fine = Integradores.integrar_Cauchy(esquema, U0, 2 * N_t, t0, tf, F)
    finer = Integradores.integrar_Cauchy(esquema, U0, 4 * N_t, t0, tf, F)

    difference_coarse_fine = np.linalg.norm(coarse - fine[::2])
    difference_fine_finer = np.linalg.norm(fine[::2] - finer[::4])
    orden = np.log(difference_coarse_fine / difference_fine_finer) / np.log(2)
    error = Richardson.error_richardson(coarse, fine[::2], orden)
    return coarse, error, orden
```

- **Líneas 14–16:** tres resoluciones del mismo problema, con N, $2N$ y $4N$ pasos.
- **Líneas 18–19:** `fine[::2]` se queda con una fila de cada dos y `finer[::4]` con una de cada cuatro, para comparar las tres en los mismos instantes. `norm` resume toda la tabla de diferencias en un único número.
- **Línea 20:** el orden, $q = \log(\text{cociente})/\log 2$.
- **Línea 21:** el error con Richardson, usando el orden *medido*. La línea comentada en el ejemplo (`ordenes = {"EE":1,...}`) era la alternativa de usar el teórico.
- **Línea 22:** devuelve la solución *gruesa* junto con el error estimado del *fino*. Para el error de la gruesa habría que multiplicarlo por 2^q (ver Fig. 9).

4.8. `Graphic.py`: los dibujos

```
import matplotlib.pyplot as plt

def grafico_sol(soluciones: dict):
    fig, ax = plt.subplots()
    for esquema, solucion in soluciones.items():
        ax.plot(solucion[:, 0], solucion[:, 1], label=esquema)
    ax.axhline(0, color="black", linewidth=0.8)
    ax.axvline(0, color="black", linewidth=0.8)
    ax.scatter([1, -1], [0, 0], color="black", zorder=5)
    ax.annotate("(1, 0)", (1, 0), xytext=(6, 6), textcoords="offset points")
    ax.annotate("(-1, 0)", (-1, 0), xytext=(6, 6), textcoords="offset points")
    ax.axis("equal")
    ax.set_xlabel("Posición x")
    ax.set_ylabel("Velocidad v")
    ax.set_title("Órbita del oscilador armónico simple")
    ax.legend()
    return fig

def grafico_error(tiempos: list, errores: dict):
    fig, ax = plt.subplots()
    for esquema, error in errores.items():
        ax.plot(tiempos, error[:, 0], label=f"Error en x ({esquema})")
        ax.plot(tiempos, error[:, 1], label=f"Error en v ({esquema})")
    ax.set_xlabel("Tiempo")
    ax.set_ylabel("Error")
    ax.set_title("Error temporal estimado mediante Richardson")
    ax.legend()
    return fig
```

No hay matemáticas nuevas, solo dibujo.

- `grafico_sol` pinta una órbita por método. Marca $(1, 0)$, el punto de partida, y $(-1, 0)$, el otro extremo del vaivén, adonde el columpio llega en $t = \pi$.
- `grafico_error` pinta el error estimado frente al tiempo, para posición y velocidad, en cada método.
- Ambas reciben diccionarios {método: tabla}, así que valen para cualquier número de métodos.

4.9. `Ej_uso_Esquemas.py`: el experimento final

```
1  """
2  Ejemplo de esquemas de integración temporal para el oscilador armónico simple.
3  El error temporal de cada método se estima con Richardson, comparando los
4  resultados con pasos h y h/2.
5  """
6  import matplotlib.pyplot as plt
7  from numpy import abs, array, linspace
8  from Integradores import integrar_Cauchy
9  from Richardson import error_richardson
10 from Esquema_fisico import oscilador as F
11 import Cauchy_Problem
12 import Graphic
13
14 N_t = 100 #Numero de pasos de tiempo para la integración
15 t_i = 0
16 t_f = 10
17 U0 = array([1, 0])
18 #ordenes = {"EE": 1, "EI": 1, "CN": 2, "RK4": 4}
19 esquemas = ["EE", "EI", "CN", "RK4"]
20 soluciones = {}
21 errores = {}
22 for esquema in esquemas:
23 #for esquema, orden in ordenes.items():
24     coarse, error, orden = Cauchy_Problem.sol_error_cauchy(esquema, U0, N_t, t_i, t_f, F)
25     print(f"Esquema: {esquema}, Orden estimado: {orden:.2f}")
26     soluciones[esquema] = coarse
27     errores[esquema] = error
28
29 tiempos = linspace(t_i, t_f, N_t + 1)
30
31 Graphic.grafico_sol(soluciones)
32 Graphic.grafico_error(tiempos, errores)
33 plt.show()
```

- **Líneas 22–27:** para cada método, resolver, medir el orden, estimar el error y guardar los resultados en diccionarios (“carpetas con etiqueta”).
- **Línea 29:** las marcas de tiempo para el eje horizontal, las mismas que crea `partition`.
- **Líneas 8–9:** importa `integrar_Cauchy`, `error_richardson` y `abs`, pero no los usa.

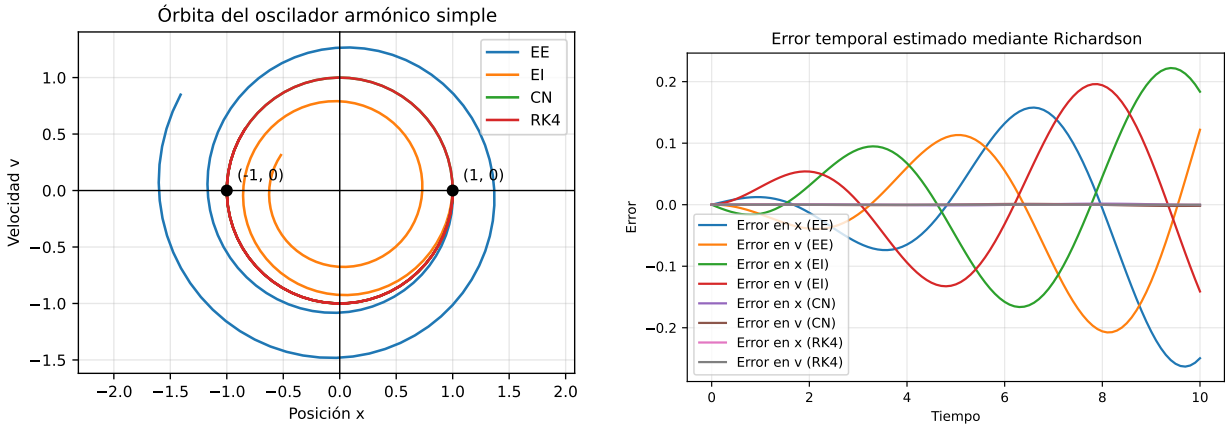


Figura 8: Resultado de `Milestone_3/Ej_uso_Esquemas.py`. Izquierda: EE se escapa, EI se hunde, y CN y RK4 se quedan en el círculo (RK4 ya no es igual a EE). Derecha: errores estimados. Los de EE y EI (~ 0.2) tapan a los de CN ($\sim 10^{-3}$) y RK4 ($\sim 10^{-6}$); con escala logarítmica se verían todos.

5. Resumen: la personalidad de cada método

Método	Cómo decide el paso	Orden	Radio en $t = 10$	Atraso en $t = 10$	Personalidad física
Exacto	gira Δt	–	1	0	columpio real
EE	dirección al *salir*	1	1.645	0.033	empujón fantasma: sube cada vez más
EI	dirección al *llegar*	1	0.608	0.033	rozamiento fantasma: se va parando
CN	media de salir y llegar	2	1.000	0.008	columpio perfecto, reloj que atrasa un poco
RK4	explorador con 4 miradas	4	0.9999993	0.000008	casi indistinguible del real

(Valores con $\Delta t = 0.1$, 100 pasos. "Atraso" es el ángulo, en radianes, que se ha quedado atrás.)

- **Explícito o implícito.** Los explícitos (EE, RK4) calculan el paso directamente: son baratos, pero pueden explotar si el paso es grande. Los implícitos (EI, CN) tienen que resolver una ecuación en cada paso, que es más caro, pero a cambio son mucho más estables.
- **Ajustar a ciegas o con manual.** El punto fijo (ajuste a ciegas) es sencillo pero lento, y falla con pasos grandes. Newton (con manual) necesita la sensibilidad, pero converge rapidísimo; con el muelle, en un solo salto.
- **Medir sin saber la respuesta.** Comparando resoluciones con pasos Δt, $\Delta t/2$ y $\Delta t/4$ se mide el orden y se estima el error sin conocer la solución exacta (Milestone 3).

Apéndice: errores encontrados en el código

1. `Milestone_2/Esquema_fisico.py`, línea 1: `from array import array` trae la `array` equivocada y la función falla. *Corregido en M3.*
2. `Milestone_2/Esquemas_temporales.py` y `Esquemas_temporales_2.py`: en RK4 el explorador no se mueve (se suma el avance a t en vez de a $\mathbf{U}$), así que RK4 es igual que EE. *Corregido en M3.*
3. `Milestone_2/Esquemas_temporales_2.py`: `My_Gauss_Seidel` usa una variable antes de crearla y compara al revés. *Corregido en M3 (`Linear_Solvers.py`).*

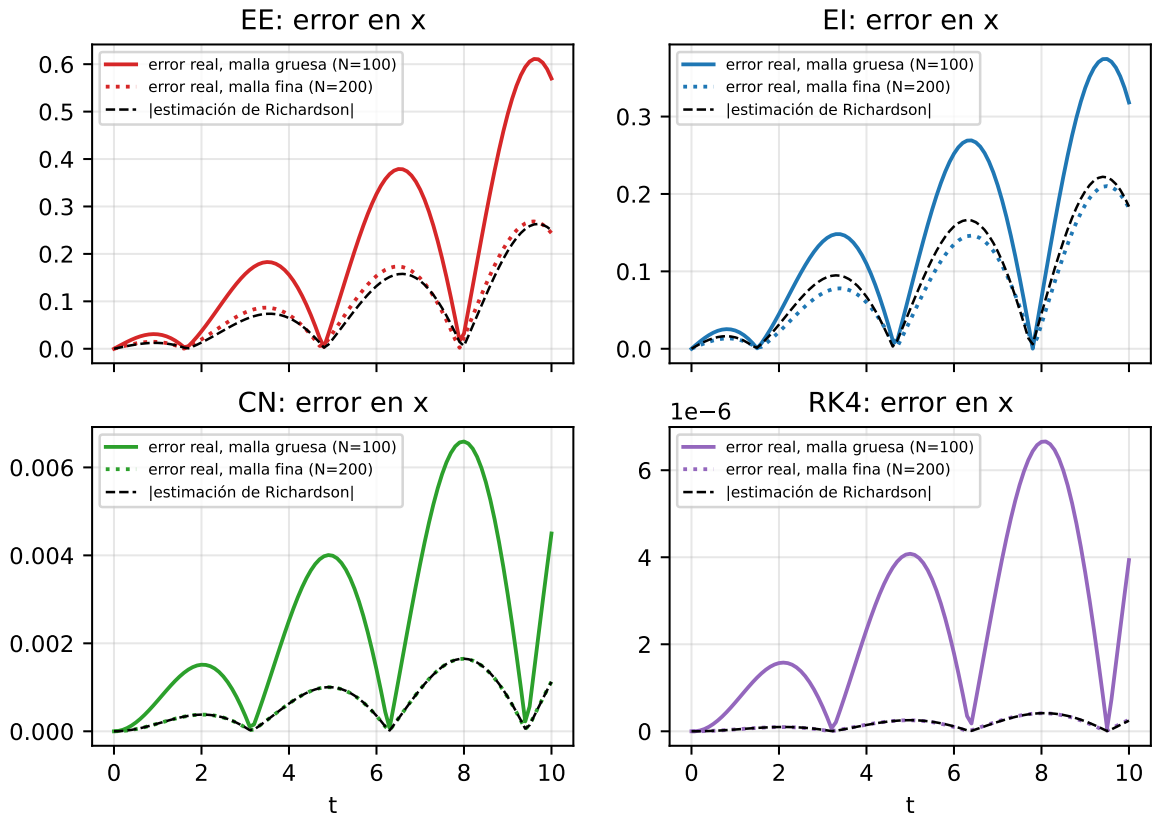


Figura 9: El examen, que solo es posible porque conocemos la solución exacta del muelle. La estimación de Richardson (rayas negras) sigue al error real de la solución *fina* (punteado). El error de la solución gruesa (línea continua) es unas 2^q veces mayor. En CN y RK4 el acierto es excelente; en EE y EI es aproximado, por lo explicado sobre el orden.

4. `Milestone_1/Problema_2_2.py`: los comentarios dicen "punto medio implícito", pero la fórmula es la del trapecio (CN). Para el muelle coinciden; en general no.

5. `Milestone_3/Cauchy_Problem.py`: devuelve la solución gruesa con el error del fino. Para el error de la gruesa hay que multiplicar por 2^q, o bien devolver `fine[::2]`.

6. `Milestone_3/Linear_Solvers.py`: es una iteración de punto fijo, no Gauss–Seidel, y no se usa.

7. Detalles menores: nombre de método desconocido con error poco claro en `Milestone_2/Integradores.py`; `N: float` debería ser `int`; importaciones y cuadernos sin usar en los ejemplos; el fallo se calcula dos veces por vuelta en `Problema_2_2.py`; el millón de pasos de `Problema_2.py` tarda unos 10 s.